

Attributing Wall-Clock Speedup in Draft Model Dimensions for Knowledge-Distilled Speculative Decoding

Yuchen Ouyang | 388166 | yuchen.ouyang@epfl.ch
Zhiyan Ke | 407061 | zhiyan.ke@epfl.ch
Tianyi Zhu | 386499 | tianyi.zhu@epfl.ch
Siyuan Zhang | 388143 | siyuan.zhang@epfl.ch
ShallowSeek

1 Progress and Preliminary Results

We implemented a complete speculative decoding (SD) pipeline for target–draft language model pairs, including KD-based draft training, SD inference, and automatic evaluation of acceptance rate, average accepted tokens, wall-clock speedup, and throughput. We evaluate a pre-trained draft and drafts trained with Forward KL (FKL), Reverse KL (RKL), and Jensen–Shannon Divergence (JSD). Unless otherwise specified, we use Qwen2.5-3B-Instruct as the target and Qwen2.5-0.5B-Instruct as the draft.

We further conduct ablations on dataset scale (UltraChat-10k vs. 50k), KD temperature ($T = 1$ vs. $T = 2$), loss weight ($\alpha = 0.5$ vs. 1.0), and target size (Qwen3-14B/0.6B vs. Qwen3-8B/0.6B)¹. Runtime generation parameters are fixed as SD block size $\gamma = 4$, sampling temperature $\tau = 1$, and $\text{top}_p = 0.9$. Detailed results are in the appendix.

2 Key Findings

Knowledge distillation improves draft quality. Under the default configuration ($\alpha = 1.0$, $T = 1.0$, UltraChat-50k), all distilled drafts outperform the pretrained baseline in acceptance rate and average accepted tokens per verification step.

RKL achieves the strongest acceptance performance. Among the objectives under the default configuration, RKL achieves the highest acceptance rate, while JSD gives the highest throughput.

Distillation temperature. The best KD temperature depends on the divergence objective. Increasing T from 1 to 2 improves FKL in acceptance rate and speedup, but substantially degrades RKL and JSD, suggesting different sensitivities to teacher-distribution smoothness.

Loss weight. Reducing the loss weight from

$\alpha = 1.0$ to $\alpha = 0.5$ introduces CE loss in addition to KL loss. RKL+CE and JSD+CE show lower acceptance rates, suggesting that emphasizing the original data distribution can weaken fitting to the target model distribution. In contrast, FKL is only weakly affected by adding CE.

UltraChat-10k vs. UltraChat-50k. Increasing the training set from 10k to 50k improves draft quality across KD losses. RKL benefits the most, while the gains for FKL and JSD are relatively small.

Qwen3-14B/0.6B vs. Qwen3-8B/0.6B. Compared with the 8B target, the 14B target leads to higher training and evaluation losses and lower acceptance rates, consistent with the increased difficulty of imitating a larger model. The same loss ranking is observed across this separate model family. Despite lower acceptance rates, the larger target can still yield higher wall-clock speedup because the autoregressive baseline is slower.

3 Challenges

Limited draft–target size gap. The 0.5B draft and 3B target have a small computational gap, limiting the wall-clock gains.

Strong autoregressive baselines. KV caching, fused kernels, and optimized attention reduce the remaining acceleration space.

Runtime overhead. Verification, cache management, synchronization, and rejection handling add latency beyond what acceptance statistics capture.

4 Plan for Final Submission

In summary, our overall plan remains unchanged. The remaining work will focus on these directions:

Data-source ablation, part II. We plan to train the draft model using responses generated by the target model, and compare the performance with the model trained on the original UltraChat dataset.

Runtime configuration sweep. We will conduct a runtime sweep on γ and max_new_tokens to understand the factors affecting wall-clock performance.

¹We do not use the same Qwen2.5 family because Qwen2.5-7B-Instruct does not share the same tokenizer as Qwen2.5-3B-Instruct and Qwen2.5-0.5B-Instruct.

Table 1: Speculative decoding performance under different loss weights ($\alpha \in \{0.5, 1.0\}$).

KD Loss	$\alpha = 1.0$			$\alpha = 0.5$		
	Acc. Rate	Avg. Acc. Tok.	Speedup	Acc. Rate	Avg. Acc. Tok.	Speedup
Pretrained	0.452	1.807	0.499	0.452	1.807	0.499
FKL	0.451	1.802	0.496	0.451	1.802	0.501
RKL	0.471	1.883	0.520	0.465	1.860	0.519
JSD	0.470	1.880	0.512	0.437	1.747	0.494

Table 2: Speculative decoding performance under different KD temperatures ($T \in \{1, 2\}$) on UltraChat-50k.

KD Loss	T=1			T=2		
	Acc. Rate	Avg. Acc. Tok.	Speedup	Acc. Rate	Avg. Acc. Tok.	Speedup
Pretrained	0.450	1.801	0.508	0.450	1.801	0.508
FKL	0.451	1.804	0.509	0.461	1.845	0.519
RKL	0.472	1.888	0.521	0.451	1.806	0.507
JSD	0.471	1.886	0.524	0.461	1.843	0.512

Table 3: Speculative decoding performance under different training data sizes on UltraChat.

KD Loss	UltraChat 50k			UltraChat 10k		
	Acc. Rate	Avg. Acc. Tok.	Speedup	Acc. Rate	Avg. Acc. Tok.	Speedup
Pretrained	0.450	1.801	0.508	0.450	1.801	0.508
FKL	0.451	1.804	0.509	0.447	1.788	0.509
RKL	0.472	1.888	0.521	0.463	1.851	0.508
JSD	0.471	1.886	0.524	0.469	1.876	0.524

Table 4: Speculative decoding performance under different target model sizes.

KD Loss	Qwen3-14B			Qwen3-8B		
	Acc. Rate	Avg. Acc. Tok.	Speedup	Acc. Rate	Avg. Acc. Tok.	Speedup
Pretrained	0.377	1.509	0.452	0.402	1.607	0.435
FKL	0.354	1.414	0.433	0.383	1.533	0.424
RKL	0.402	1.607	0.467	0.420	1.682	0.447
JSD	0.392	1.567	0.462	0.418	1.671	0.445

²This experiment (Table 1) run on the RTX4090 on another cluster instead of RunAI pod. So the baseline metric values differ from the other ones.

³All speedup ratios are smaller than 1: The baseline is vanilla decoding with only target model generation. Due to CUDA system issues, our implementation of speculative decoding was poorly optimized compared to optimized vanilla decoding. Therefore, the absolute value of speedup is lower than expected, instead the relative value is the focus for our project.

We tried to use vLLM for inference acceleration, but the current version installed on cluster does not support "draft" mode in speculative decoding.